



MERRIMACK COLLEGE

CSC 6033

Week 8

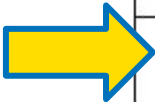
Decidability

Languages, Automata, and Decidability - Dr. Paulo Fernandes

Timeline

Where are we?

Unit	Week	Topic	Projects	Quizzes	In-class Exercises	Exam
1	1	From Python to C++	P#1	Q#1	E#1	
1	2	Developing with C++	P#2	Q#2	E#2	
2	3	Finite Automata	P#3	Q#3	E#3	
2	4	Regular Expressions	P#4	Q#4	E#4	
3	5	Pushdown Automata	P#5	Q#5	E#5	
4	6	Turing Machines	P#6	Q#6	E#6	
5	7	Processing Input and Output	P#7	Q#7	E#7	
6	8	Decidability				Final Exam (all units)



MERRIMACK COLLEGE

All deadlines are hard now in the end of the course.

Presentation Agenda

Week 8

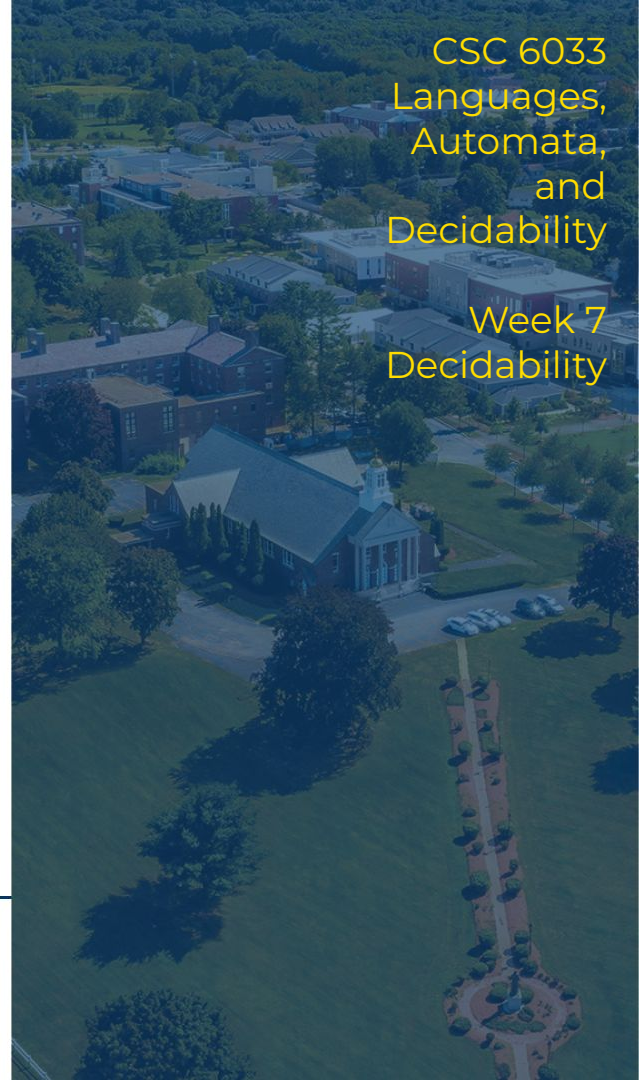
- Decidability
 - a. Turing Machines that eventually stop
 - b. Complexity and Decidability
 - c. The halting problem
- Final Steps
 - a. Final Exam
 - b. Grades
 - c. After this course



MERRIMACK COLLEGE

CSC 6033
Languages,
Automata,
and
Decidability

Week 7
Decidability



Decidability

“Just do it!”

Nike advertisement campaign

A Turing Machine is capable of great many things. However, it cannot be taken for granted that a Turing Machine will eventually end its processing.

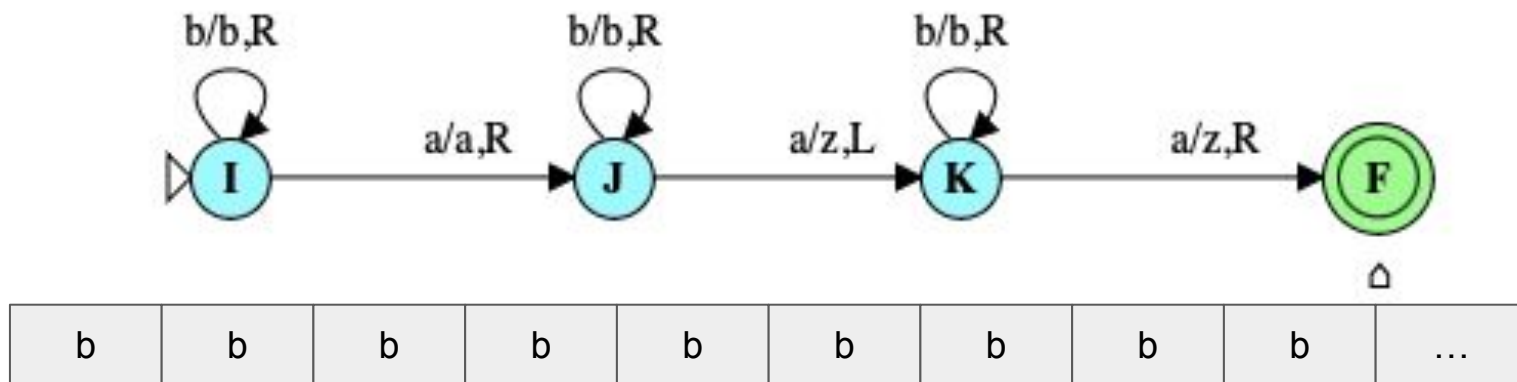
The examples we saw either end with the acceptance reaching one final state, or with the rejection if a symbol unexpectedly makes the TM to stuck in a non final state.

- **Turing Machines that eventually stop**
- Complexity and Decidability
- The halting problem



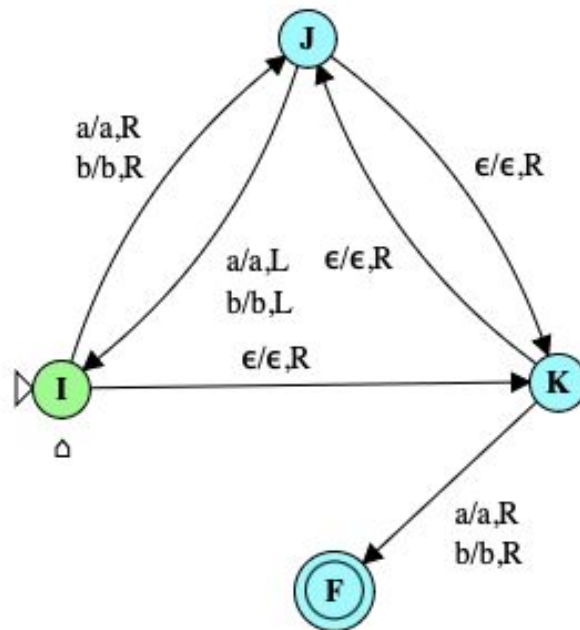
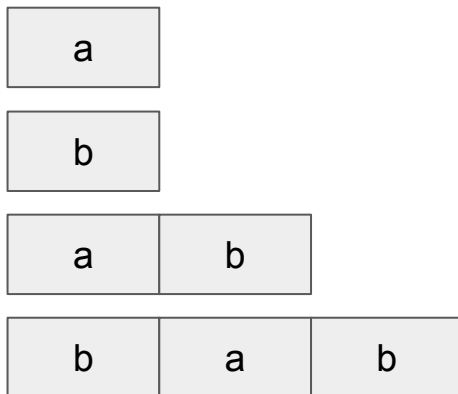
Turing Machines that are likely to stop

For example, the TM below accepts words where the first letter **a** has to be followed by another letter **a**. This TM will stop, unless there is at some point before a double **a**, an infinite sequence of letters **b**. Therefore, it will eventually stop for finite input words.



Turing Machines that are unlikely to stop

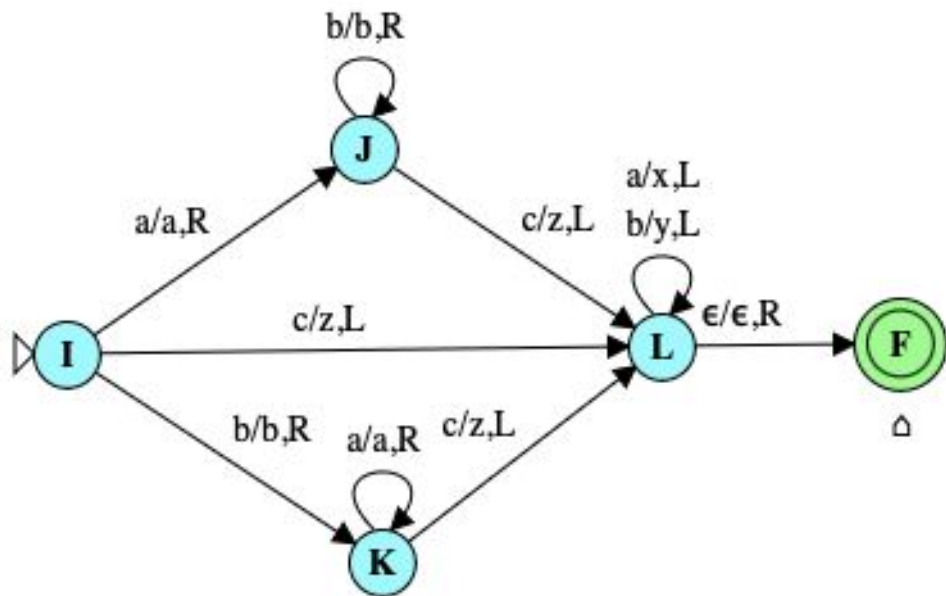
For example, this TM does not accept words (unless it starts with an empty symbol and followed by a letter **a** or **b**).



Turing Machines that eventually stop

Is this TM likely to stop or not?

- What are the words this TM accepts?
- What is the output of this TM?
- Can you find a word that lead to a stop?
- Can you find a word that does not lead to a stop?



Decidability

“Just do it!”

Nike advertisement campaign

A Turing Machine is capable of great many things. However, it cannot be taken for granted that a Turing Machine will eventually end its processing.

The examples we saw either end with the acceptance reaching one final state, or with the rejection if a symbol unexpectedly makes the TM to stuck in a non final state.

- Turing Machines that eventually stop
- **Complexity and Decidability**
- The halting problem

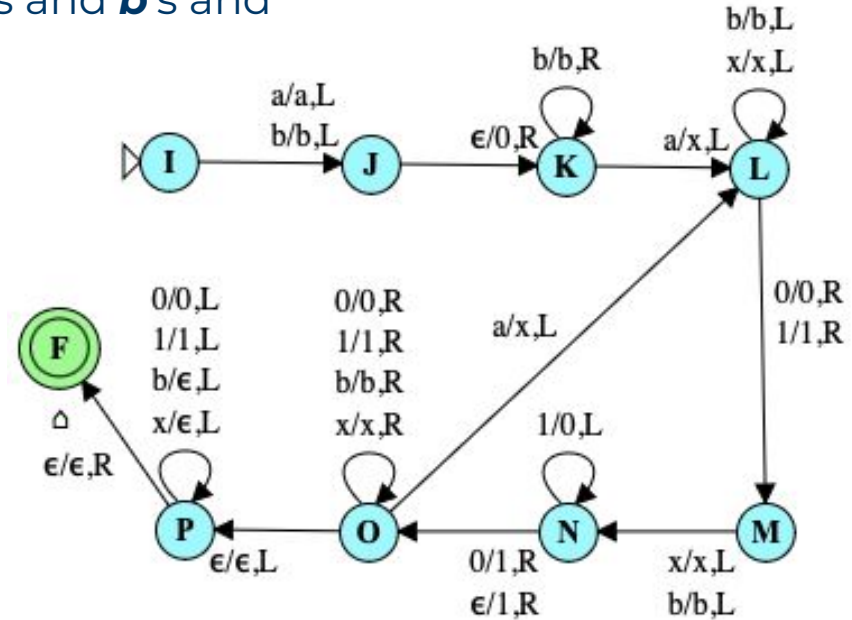


TM and Programming Languages

This TM receives a word composed by **a**'s and **b**'s and counts the number of letters **a** in binary.

This C++ code does the same.

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string str;
6     int count = 0;
7     char buffer [1000];
8
9     std::cout << "Enter a string: ";
10    std::cin >> str;
11    for (char c : str) {
12        if (c == 'a')
13            count++;
14    }
15    std::string binary = "";
16    while (count > 0) {
17        binary = std::to_string(count % 2) + binary;
18        count /= 2;
19    }
20    std::cout << binary << std::endl;
21    return 0;
22 }
```



Complexity

The solution is a brute force algorithm, thus, we have to account for:

- a for loop in line 11 going as many times as the number of characters:
 - linear to the number of characters (***n***);
- a loop at line 16 going as $\log_2$ of the number of characters ***a*** (***m***):
 - logarithmic to the a number smaller than the number of characters (***m*** < ***n***).

$O(n)$

```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string str;
6     int count = 0;
7     char buffer [1000];
8
9     std::cout << "Enter a string: ";
10    std::cin >> str;
11    for (char c : str) {
12        if (c == 'a')
13            count++;
14    }
15    std::string binary = "";
16    while (count > 0) {
17        binary = std::to_string(count % 2) + binary;
18        count /= 2;
19    }
20    std::cout << binary << std::endl;
21    return 0;
22 }
```



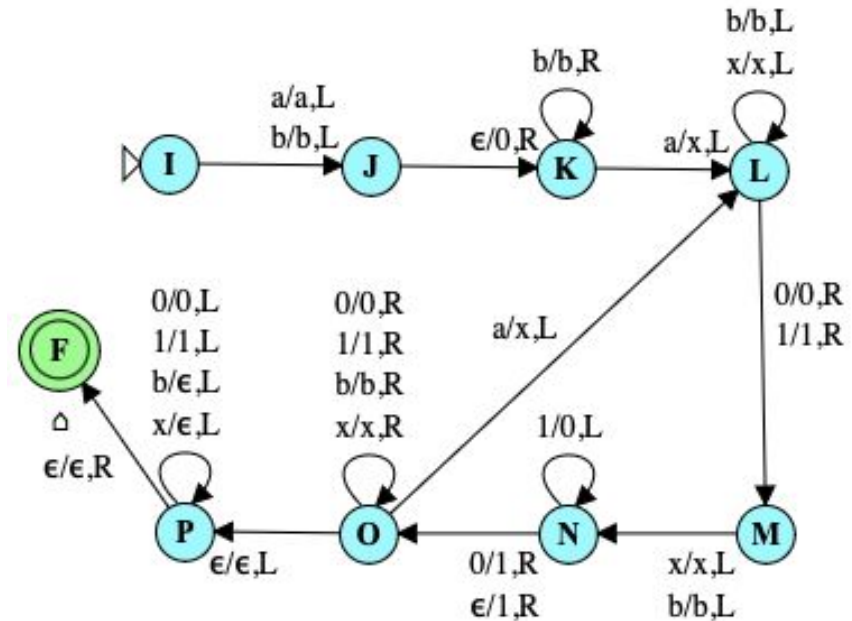
Complexity

The TM has to go over the n characters as many times as the number of letters a (m).

The passages back and forward will represent m times up to n movements of the tape.

Thus, this solution will be upper bounded as the product of n and m .

$$O(nm)$$

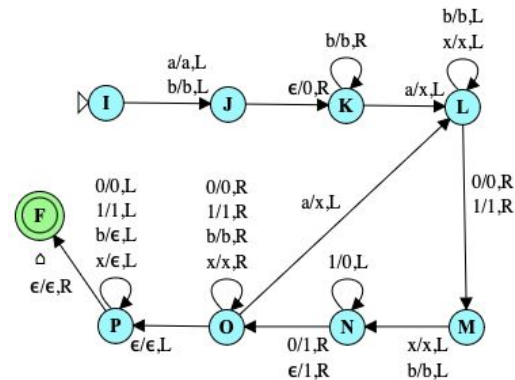


Complexity and Decidability

The complexity may not be the same, since the algorithms (and the basic instructions) may not be the same.

However, can these solutions have the same behavior in terms of decidability.

- Will both solutions certainly end?
- Can one end and the other don't?
- Is decidability linked to a problem or to a solution (algorithm)?



```
1 #include <iostream>
2 #include <string>
3
4 int main() {
5     std::string str;
6     int count = 0;
7     char buffer [1000];
8
9     std::cout << "Enter a string: ";
10    std::cin >> str;
11    for (char c : str) {
12        if (c == 'a')
13            count++;
14    }
15    std::string binary = "";
16    while (count > 0) {
17        binary = std::to_string(count % 2) + binary;
18        count /= 2;
19    }
20    std::cout << binary << std::endl;
21    return 0;
22 }
```



Decidability

“Just do it!”

Nike advertisement campaign

A Turing Machine is capable of great many things. However, it cannot be taken for granted that a Turing Machine will eventually end its processing.

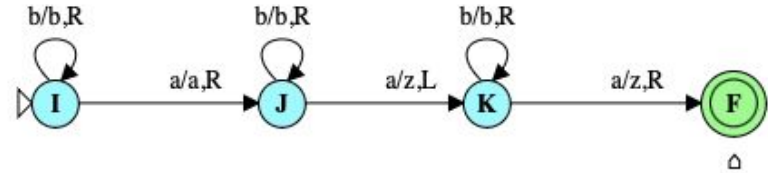
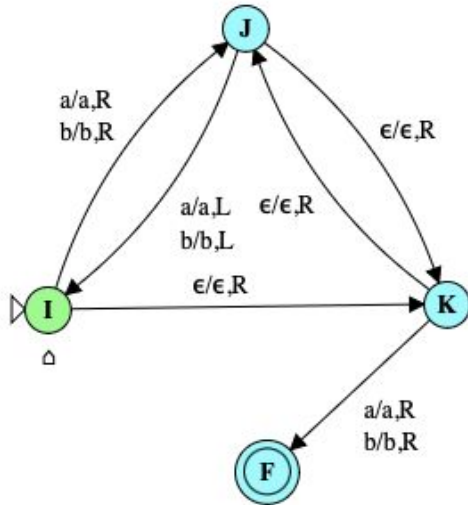
The examples we saw either end with the acceptance reaching one final state, or with the rejection if a symbol unexpectedly makes the TM to stuck in a non final state.

- Turing Machines that eventually stop
- Complexity and Decidability
- **The halting problem**



Does a given TM (or a program) ends?

Sometimes we can answer that clearly.



```
1 #include <iostream>
2
3 int main()
4 {
5     int i = 0;
6     while (i < 20) {
7         i++;
8     }
9     return 0;
10 }
```

```
1 #include <iostream>
2
3 int main()
4 {
5     int i = 0;
6     int j = 1;
7     while (i < 2) {
8         i += j;
9         j *= -1;
10    }
11    return 0;
12 }
```



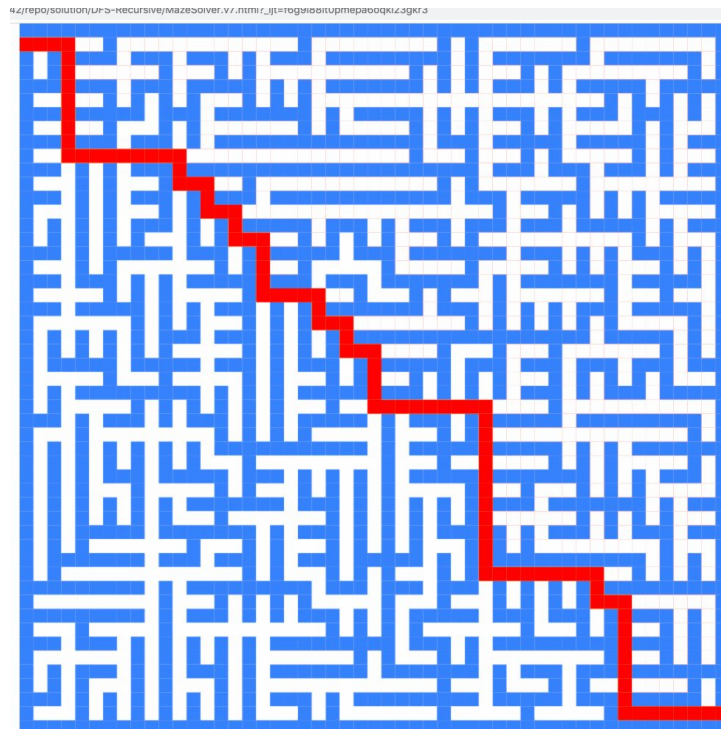
The Halting Problem

In a general case, it is impossible to be certain.

In fact, the halting problem is undecidable even in a simpler form:

Given a TM (or a program for that matter) and a specific input sequence, it is not known if this TM (or program) will eventually end.

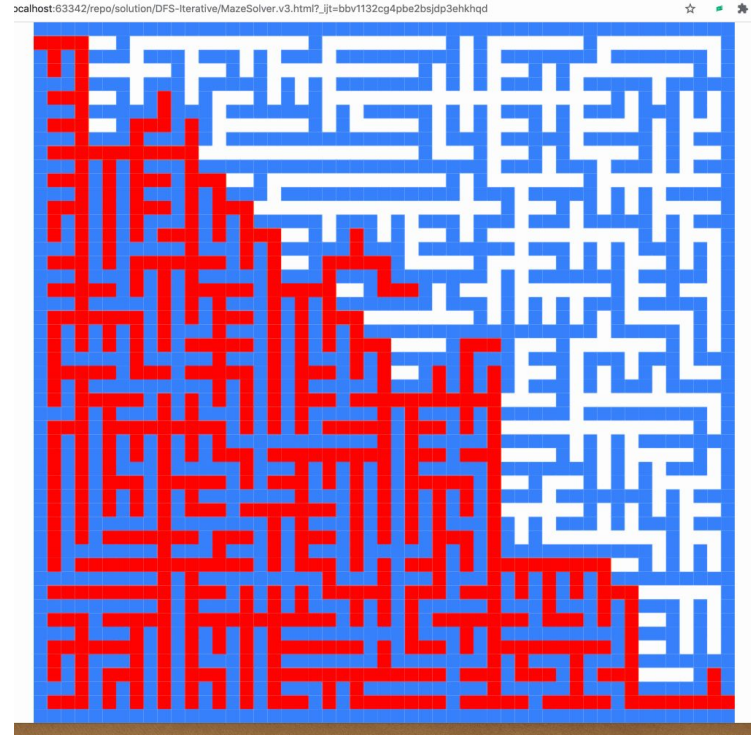
Obviously, if you cannot answer it for a specific input, it is also impossible for all possible inputs.



Decidability

Practical options can cope with this serious limitation of computing.

- One way to verify it, is to wait until a given time threshold and then give up after it:
 - The phone call wait dilemma;
- Another way is to keep track of the current state, and if the solution returns to a previous state, give it up:
 - The cycle detection approach.

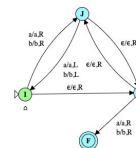


Limits of Computation

While the the practical shortcuts can give some solutions, it remains some problems that cannot be fully solved, and yet many of the problems are...

Non-exhaustive list of undecidable problems:

- The halting problem
 - will a TM halt for a given input?
- Determining if a context-free grammar is ambiguous
 - does a CFG grammar generates a same word with more than one derivation?
- The three-body problem
 - given three objects of similar masses close enough to gravitationally affect each other, can we analytically compute their trajectories?



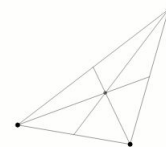
- $\Sigma = \{a, b\}$
- $NT = \{S\}$

Productions:

P1 $S \Rightarrow aS$

P2 $S \Rightarrow Sb$

P3 $S \Rightarrow \varepsilon$



MERRIMACK COLLEGE

Video: [The Boundary of Computation](#).

Next Steps

“... and now, the end is near...”

My Way lyrics

- **The Final Exam**
- Grades
- After this course



The Final Exam

Go to Module 8 on Canvas and take the Final Exam.

- The exam is composed by 8 questions;
- You have 4 hours once you start to take the exam (The final exam is to be taken in one sitting);
- You will download a **.pdf** with the 8 questions, and within 4 hours you need to upload a single **.pdf** with your answers;
- There is about one question about each week topic;
- You can consult all class materials, but not general access to the Internet;
- Prepare yourself before (don't count on learning during the exam).



MERRIMACK COLLEGE

The exam should be available for you to take from tomorrow until this Saturday.

The Final Exam

General topics for the 8 questions of the exam:

1. Basic C++ programming language commands;
2. C++ programming language development;
3. Finite State Automata;
4. Regular Expressions;
5. Pushdown Automata;
6. Turing Machines (with input);
7. Turing Machines with (input and) output;
8. Decidability.



MERRIMACK COLLEGE

The exam should be available for you to take from tomorrow until this Saturday.

Next Steps

“... and now, the end is near...”

My Way lyrics

- The Final Exam
- **Grades**
- After this course



Evaluation

The evaluated tasks are weighted as such (105%):

Activity	Published grades	Percentage
Projects (7)	Evaluated from 0 to 100 each	35.0%
Quizzes (7)	Evaluated from 0 to 100 each	28.0%
In-class Exercises (7)	Evaluated from 0 to 100 each	21.0%
Final Exam	Evaluated from 0 to 100	21.0%



MERRIMACK COLLEGE

Final deadline on Friday of the course's last week

Evaluation

The final letter grade is computed according to this table:

A	A-	B+	B	B-	C+	C	C-	F
95 and up	90 to 94.9	87 to 89.9	83 to 86.9	80 to 82.9	77 to 79.9	73 to 76.9	70 to 72.9	69.9 and low

There are no grade D+, D, or D- in graduate courses. Only C or above are passing grades. An average grade B is required to the Masters of Science in Computer Science, lower than that puts you in probation.



MERRIMACK COLLEGE

Final grades likely published first Monday after the course's end, no later than next Tuesday.

Next Steps

“... and now, the end is near...”

My Way lyrics

- The Final Exam
- Grades
- **After this course**



After this course



- CSC 6301 - Software Design and Documentation
 - Software documentation (Python and Java) and Software Engineering methodologies;
- CSC 6302 - Database Principles
 - Database concepts and usage (SQL and integration);
- CSC 6303 - Systems and Languages Survey
 - Several programming languages and operating systems;
- CSC 6304 - Advanced Programming Concepts
 - API development style, web and mobile development;
- CSC 6400 - Capstone Project (optional for most)
 - Industry or research oriented project.



MERRIMACK COLLEGE

These courses be taken two at time (any order).

” **Week 8 of CSC 6033**

- **Be ready and take the Final Exam until this Saturday;**
- **Final Office hours this Thursday;**

It was my pleasure to be your instructor for this course.

See you, when I see you!



MERRIMACK COLLEGE